

Day 1 종합 실습 결과 보고서

과제명 : 데이터 수집 미니 파이프라인 종합 실습
제출자 : 울산 4반 1조 U109 강지수

1. 실행결과 화면 캡처

1.1. CSV·Parquet 읽기/쓰기 성능 측정 결과 출력

```
(venv) → Day_1_Data_collection_mini_pipeline git:(main) python main.py
weather: 72 valid rows
ip: 1 valid row
country: 1 valid row

Read/Write benchmark (seconds):
operation      read      write
dataset format
country csv      0.000340  0.000326
           parquet 0.000727  0.000574
ip csv        0.000418  0.000560
           parquet 0.000983  0.000893
weather csv    0.001168  0.004883
           parquet 0.037643  0.035305

(ip/country datasets have only 1 row, so their timing mostly reflects fixed overhead, not format scaling.)
```

1.2. pytest 결과 화면 (pytest_result.txt)

```
reports > pytest_result.txt
1 ..... [100%]
2 13 passed in 0.04s
3 |
```

1.3. ruff 결과 화면(ruff_result.txt)

ruff_result.txt	결과 요약		
<pre>1 I001 [*] Import block is un-sorted or un-formatted 2 --> main.py:11:1 3 4 11 / import asyncio 5 12 import pandas as pd 6 13 7 14 from pipeline.benchmark import run_benchmark 8 15 from pipeline.config import DATA_DIR, REPORTS_DIR 9 16 from pipeline.fetch import fetch_all 10 17 from pipeline.schemas import CountryInfo, IPInfo 11 18 from pipeline.transform import transform_country, transform_ip, transform_weather 12 13 </pre>	규칙	의미	발생 수
	I001	import 블록이 정렬/포맷되지 않음	6
	E401	한 줄에 여러 import	1
	E501	줄 길이 100자 초과	4

2. Code 분석 결과에 대한 본인 의견

2.1.Import 구문 컨벤션 관련

Import 구문에 대한 코드 컨벤션이 별도로 존재함을 확인함. 기존에는 해당 컨벤션의 존재 여부를 인지하지 못한 상태에서 줄 간격을 최소화하거나 한 줄에 다수의 모듈을 import 하는 방식으로 작성해왔음. 분석 결과 import 모듈의 분류에 따라 작성 순서 및 방식을 구분해야 함을 확인하였으며, 향후 작업 시 이를 반영할 필요성을 느낌.

2.2. ruff check 자동 수정 기능

ruff check시, -fix 옵션을 적용할 경우, 수정 가능한 항목에 대해 자동으로 수정이 이루어짐을 확인함. 해당 기능은 추후 협업 및 코드 리뷰 작업에서 큰 도움이 될 것으로 판단됨.

2.3. 주석 작성 방식 관련

인라인 주석이 과도하게 길거나 한 줄에 지나치게 많은 내용을 포함할 경우, 오류로 분류됨을 확인함. 이에 주석은 간결하게 요약하여 작성하거나, 부득이하게 내용이 길어질 경우, 줄바꿈을 활용하여 가독성을 확보하는 방식으로 개선이 필요하다고 판단.

2.4. 종합 의견

분석을 통해 그동안 인지하지 못했던 코드 컨벤션 다수를 확인할 수 있었음. 향후 협업 진행 혹은 코드 리뷰 제출 이전 단계에서 ruff check를 습관화하는 방향이 바람직하다고 판단됨.

3. 추가 의견

3.1. 개선 사항

CSV와 Parquet의 읽기·쓰기 성능을 비교 평가한 결과, 두 항목 모두에서 CSV가 Parquet 대비 우수한 성능을 나타냄. 특히 타 2개 API 대비 데이터 수가 상대적으로 많은 Open-Meteo 데이터의 경우, 두 포맷 간 성능 격차가 더욱 크게 나타나는 것으로 확인됨. 이는 현재 데이터 규모가 상대적으로 소규모이기 때문이라 판단되며, 대규모 데이터 처리 환경에서는 Parquet이 더 유리한 결과를 보였을 것으로 예상됨.

추가로, 성능 측정 시 time 라이브러리 외에 timeit 라이브러리도 활용해볼 것.

3.2. 코드 품질 개선 측면

각 API별 데이터 수집 과정에서 리팩토링 방식에 대해 검토해볼 것. 현재는 공통 로직은 최대한 공통 함수로 통합하고, API 이름만 다른 부분은 편의상 별도로 분리하여 작성하였으나, 결과적으로 코드 구조가 깔끔하지 않다고 판단됨. 리팩토링 관련 부분을 학습할 것.